

🔍 AI-Powered PDF Compliance Scanner

Complete Step-by-Step Implementation Guide (100% Free Stack)

Free API Strategy: This guide replaces paid Claude API with **Groq** (free tier — fast Llama 3 / Mixtral inference) and optionally **Ollama** (fully local). Deployment is on **Streamlit Community Cloud** (free). No credit card needed.

📑 TABLE OF CONTENTS

- 1. Prerequisites & Free Accounts Setup
- 2. GitHub Repository Setup
- 3. Full Project Structure
- 4. Environment & Dependencies
- 5. Free AI API Setup (Groq)
- 6. Core Pipeline — State & Config
- 7. PDF Ingestion Node (PyMuPDF)
- 8. AI Compliance Nodes
- 9. LangGraph Orchestration
- 10. Database & Storage Layer
- 11. Report Generation Engine
- 12. Streamlit UI — All Pages
- 13. Configuration Files
- 14. Testing Suite
- 15. Docker Setup
- 16. GitHub Actions CI/CD
- 17. Deployment to Streamlit Cloud (Free)
- 18. Alternative: Deploy to Hugging Face Spaces (Free)
- 19. Troubleshooting & Common Errors

1. Prerequisites & Free Accounts Setup

1.1 What You Need to Install Locally

```
# Check versions — install if missing
python --version      # Need 3.11+
git --version         # Any recent version
docker --version      # Optional but recommended
```

Install Python 3.11+ from <https://www.python.org/downloads/>

1.2 Free Accounts to Create

Service	Purpose	Free Tier
GitHub	Code hosting + CI/CD	Unlimited public repos
Groq	Free AI API (Llama 3 / Mixtral)	14,400 requests/day FREE
Streamlit Community Cloud	Free deployment	1 public app free

Get Groq Free API Key (replaces paid Claude):

- 1. Go to <https://console.groq.com>
- 2. Sign up with Google/GitHub
- 3. Click **API Keys** → **Create API Key**
- 4. Copy and save — you only see it once
- 5. Free limits: **14,400 requests/day, 30 req/min** — more than enough

Alternative Free AI Options:

- **Google Gemini** (<https://aistudio.google.com>) — gemini-1.5-flash is FREE
- **Ollama** (<https://ollama.com>) — Run models 100% locally, zero API calls
- **HuggingFace Inference API** — Free tier available

2. GitHub Repository Setup

2.1 Create Repository on GitHub

1. Go to <https://github.com/new>
2. Repository name: pdf-compliance-scanner
3. Description: AI-powered PDF compliance scanning pipeline using LangGraph + Groq
4. Set to **Public** (required for free Streamlit Cloud deployment)
5. ☒ Check **Add a README file**
6. ☒ Add **.gitignore** → select **Python**
7. Click **Create repository**

2.2 Clone to Your Machine

```
git clone https://github.com/YOUR_USERNAME/pdf-compliance-scanner.git
cd pdf-compliance-scanner
```

2.3 Set Up Git Identity (if first time)

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

3. Full Project Structure

Run this to create all folders and files at once:

```
# Run from inside your cloned repository folder
mkdir -p app/pages app/components
mkdir -p pipeline/nodes
mkdir -p config/prompts
mkdir -p storage
mkdir -p reports
mkdir -p tests/fixtures
mkdir -p .github/workflows

# Create all Python files
touch app/__init__.py
touch app/main.py
touch app/pages/01_upload.py
touch app/pages/02_rules.py
touch app/pages/03_reports.py
touch app/components/__init__.py
touch app/components/uploader.py
touch app/components/report_card.py

touch pipeline/__init__.py
touch pipeline/state.py
touch pipeline/graph.py
touch pipeline/nodes/__init__.py
touch pipeline/nodes/ingest.py
touch pipeline/nodes/pii_detector.py
touch pipeline/nodes/confidentiality.py
touch pipeline/nodes/encoding_guard.py
touch pipeline/nodes/abuse_detector.py
touch pipeline/nodes/aggregator.py
touch pipeline/nodes/report_builder.py

touch config/__init__.py
touch config/rules.py
touch config/rules.json
touch config/prompts/pii_prompt.txt
touch config/prompts/confidential_prompt.txt
touch config/prompts/encoding_prompt.txt
touch config/prompts/abuse_prompt.txt

touch storage/__init__.py
touch storage/database.py

touch tests/__init__.py
touch tests/test_nodes.py
touch tests/test_pipeline.py
touch tests/conftest.py

touch Dockerfile
touch docker-compose.yml
touch requirements.txt
touch .env.example
touch .streamlit/config.toml
```

4. Environment & Dependencies

4.1 requirements.txt

```
# Web UI
streamlit==1.35.0
streamlit-authenticator==0.3.1

# LangGraph Orchestration
langgraph==0.2.14
langchain-core==0.2.10
langchain-groq==0.1.6

# AI Clients
groq==0.9.0
anthropic==0.28.0      # Optional: only if you later get Claude API

# PDF Processing
PyMuPDF==1.24.5      # fitz – fast, accurate PDF parser
reportlab==4.2.0      # PDF report generation

# Encoding & Language Detection
chardet==5.2.0
langdetect==1.0.9

# Data Validation
pydantic==2.7.0

# Utilities
python-dotenv==1.0.1
tenacity==8.3.0      # Retry with exponential backoff
uuid==1.30

# Testing
pytest==8.2.2
pytest-mock==3.14.0

# Security
bandit==1.7.9
```

4.2 Create Virtual Environment and Install

```
# Create virtual environment
python -m venv .env

# Activate it
source .env/bin/activate      # Mac/Linux
.venv\Scripts\activate        # Windows

# Install all dependencies
pip install -r requirements.txt
```

4.3 .env.example (copy this to .env and fill in your keys)

```
# Copy this file: cp .env.example .env
# Then edit .env with your actual values

# === PRIMARY: Groq (FREE) ===
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX

# === OPTIONAL: Google Gemini (FREE) ===
GOOGLE_API_KEY=AIzaXXXXXXXXXXXXXXXXXXXX

# === OPTIONAL: Anthropic Claude (PAID) ===
ANTHROPIC_API_KEY=sk-ant-XXXXXXXXXXXXXXXXXXXX

# === AI Provider Selection ===
# Options: "groq" | "gemini" | "anthropic" | "ollama"
AI_PROVIDER=groq

# === Groq Model Selection (all FREE) ===
# Options: llama3-70b-8192 | llama3-8b-8192 | mixtral-8x7b-32768 | gemma2-9b-it
GROQ_MODEL=llama3-70b-8192

# === App Settings ===
MAX_FILE_SIZE_MB=50
MAX_PAGES=500
DEBUG=false
```

4.4 .streamlit/config.toml

```
[theme]
primaryColor = "#FF4B4B"
backgroundColor = "#0E1117"
secondaryBackgroundColor = "#262730"
textColor = "#FAFAFA"
font = "sans serif"

[server]
maxUploadSize = 50
headless = true
```

5. Free AI API Setup (Groq)

5.1 AI Provider Factory — config/ai_provider.py

This is the key file that makes the project work with ANY free AI service:

```
# config/ai_provider.py
"""
AI Provider Factory – supports Groq (free), Gemini (free), Anthropic (paid), Ollama (local).
Switch providers by changing AI_PROVIDER in your .env file.
"""

import os
import json
from groq import Groq
from tenacity import retry, stop_after_attempt, wait_exponential
from dotenv import load_dotenv

load_dotenv()

AI_PROVIDER = os.getenv("AI_PROVIDER", "groq")
GROQ_MODEL = os.getenv("GROQ_MODEL", "llama3-70b-8192")
```

```

def get_ai_client():
    """Return the configured AI client."""
    if AI_PROVIDER == "groq":
        return Groq(api_key=os.getenv("GROQ_API_KEY"))
    elif AI_PROVIDER == "anthropic":
        import anthropic
        return anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
    elif AI_PROVIDER == "gemini":
        import google.generativeai as genai
        genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))
        return genai
    elif AI_PROVIDER == "ollama":
        # Ollama uses OpenAI-compatible API – no key needed
        from openai import OpenAI
        return OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
    else:
        raise ValueError(f"Unknown AI_PROVIDER: {AI_PROVIDER}")

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=2, max=60),
    reraise=True
)
def call_ai(system_prompt: str, user_message: str, max_tokens: int = 1024) -> str:
    """
    Universal AI call that works with Groq, Anthropic, Gemini, or Ollama.
    Returns the text response as a string.
    Includes automatic retry with exponential backoff for rate limits.
    """
    client = get_ai_client()

    if AI_PROVIDER in ["groq", "ollama"]:
        model = GROQ_MODEL if AI_PROVIDER == "groq" else os.getenv("OLLAMA_MODEL", "llama3")
        response = client.chat.completions.create(
            model=model,
            max_tokens=max_tokens,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_message}
            ],
            temperature=0.1, # Low temperature for consistent classification
        )
        return response.choices[0].message.content

    elif AI_PROVIDER == "anthropic":
        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=max_tokens,
            system=system_prompt,
            messages=[{"role": "user", "content": user_message}]
        )
        return response.content[0].text

    elif AI_PROVIDER == "gemini":
        model = client.GenerativeModel(
            model_name="gemini-1.5-flash",
            system_instruction=system_prompt
        )
        response = model.generate_content(user_message)
        return response.text

    raise ValueError(f"Provider {AI_PROVIDER} not implemented")

def parse_ison_response(raw_text: str) -> dict:

```

```

"""
Safely parse JSON from AI response.
Handles cases where the model wraps JSON in markdown code blocks.
"""

text = raw_text.strip()

# Strip markdown code blocks if present
if text.startswith("```json"):
    text = text[7:]
elif text.startswith("```"):
    text = text[3:]

if text.endswith("```"):
    text = text[:-3]

text = text.strip()

try:
    return json.loads(text)
except json.JSONDecodeError:
    # Try to find JSON object in the text
    import re
    match = re.search(r'\{.*\}', text, re.DOTALL)
    if match:
        return json.loads(match.group())
    return {"error": "Failed to parse AI response", "raw": text}

```

6. Core Pipeline — State & Config

6.1 pipeline/state.py

```

# pipeline/state.py
"""
LangGraph State Schema – typed state object that flows through every pipeline node.
"""

from typing import TypedDict, List, Optional, Dict, Any

class Flag(TypedDict):
    """A single compliance violation flag."""
    type: str          # e.g., "EMAIL", "API_KEY", "HATE_SPEECH"
    value: str          # The actual flagged content (truncated for privacy)
    context: str        # Surrounding text for context (50-150 chars)
    confidence: float    # 0.0 to 1.0
    risk_level: str      # "low" | "medium" | "high" | "critical"
    position: Optional[str] # Page location hint

class PageResult(TypedDict):
    """Compliance results for a single PDF page."""
    page_num: int
    text_preview: str      # First 200 chars of extracted text
    char_count: int
    pii_flags: List[Flag]
    confidential_flags: List[Flag]
    encoding_flags: List[Flag]
    abuse_flags: List[Flag]
    pii_risk: str
    confidential_risk: str
    encoding_risk: str
    abuse_risk: str
    overall_risk: str      # "low" | "medium" | "high" | "critical"

```

```

class PageText(TypedDict):
    """Raw extracted text data for a single page."""
    page_num: int
    text: str
    char_count: int
    detected_encoding: str
    encoding_confidence: float
    image_count: int


class PipelineState(TypedDict):
    """
    Complete pipeline state – passed through every LangGraph node.
    Nodes return partial dicts that update specific keys only.
    """
    # Input
    pdf_path: str
    pdf_name: str
    upload_id: str

    # After ingestion
    total_pages: int
    pages_text: List[PageText]

    # After compliance checks
    pii_results: List[Dict]
    confidential_results: List[Dict]
    encoding_results: List[Dict]
    abuse_results: List[Dict]

    # After aggregation
    page_results: List[PageResult]
    summary: Dict[str, Any]

    # Config
    compliance_rules: Dict[str, Any]

    # Output
    report_path: Optional[str]
    processing_complete: bool
    errors: List[str]

```

6.2 config/rules.py

```

# config/rules.py
"""
Load and save compliance rules from config/rules.json.
"""

import json
import os
from pathlib import Path

RULES_FILE = Path(__file__).parent / "rules.json"

def load_rules() -> dict:
    """Load compliance rules from JSON file."""
    if not RULES_FILE.exists():
        return get_default_rules()
    with open(RULES_FILE, "r") as f:
        return json.load(f)

```



```

def save_rules(rules: dict) -> None:
    """Save updated compliance rules to JSON file."""
    with open(RULES_FILE, "w") as f:
        json.dump(rules, f, indent=2)

def get_default_rules() -> dict:
    """Return default compliance rules if no file exists."""
    return {
        "version": "1.0",
        "sensitivity": "high",
        "pii": {
            "enabled": True,
            "detect_email": True,
            "detect_phone": True,
            "detect_ssn_aadhaar": True,
            "detect_credit_card": True,
            "detect_address": True,
            "detect_dob": True,
            "min_confidence": 0.75,
            "risk_threshold": "medium"
        },
        "confidentiality": {
            "enabled": True,
            "detect_api_keys": True,
            "detect_passwords": True,
            "detect_trade_secrets": True,
            "detect_financial_data": True,
            "detect_merger_acquisition": True,
            "custom_keywords": [
                "PROJECT TITAN", "Operation Phoenix",
                "internal use only", "confidential"
            ],
            "min_confidence": 0.70
        },
        "encoding": {
            "enabled": True,
            "require_utf8": True,
            "allowed_languages": ["en"],
            "min_encoding_confidence": 0.85,
            "flag_non_ascii": True,
            "non_ascii_threshold": 5
        },
        "abuse": {
            "enabled": True,
            "detect_hate_speech": True,
            "detect_sexual_content": True,
            "detect_violence_incitement": True,
            "detect_illegal_content": True,
            "detect_harassment": True,
            "sensitivity_level": "high",
            "zero_tolerance_categories": ["child_safety", "terrorism"]
        },
        "reporting": {
            "include_text_context": True,
            "context_window_chars": 150,
            "generate_pdf_report": True,
            "generate_json_report": True
        }
    }

```

7. PDF Ingestion Node (PyMuPDF)

7.1 pipeline/nodes/ingest.py

```
# pipeline/nodes/ingest.py
"""
PDF ingestion node – extracts text from every page using PyMuPDF.
Handles text PDFs, detects image-only pages, captures encoding metadata.
"""

import fitz # PyMuPDF
import chardet
from pipeline.state import PipelineState

def ingest_node(state: PipelineState) -> dict:
    """
    Extract text page-by-page from uploaded PDF.

    Returns:
        Partial state update with 'total_pages' and 'pages_text'.
    """
    pages_text = []
    errors = list(state.get("errors", []))

    try:
        doc = fitz.open(state["pdf_path"])

        for page_num in range(len(doc)):
            page = doc[page_num]

            # Primary extraction: direct text (fast, accurate for text PDFs)
            text = page.get_text(
                "text",
                flags=fitz.TEXT_PRESERVE_WHITESPACE | fitz.TEXT_MEDIABOX_CLIP
            )

            # Fallback: image-only page detection
            if not text.strip() and len(page.get_images()) > 0:
                text = (
                    f"[PAGE {page_num+1}: Image-only page detected. "
                    f"Contains {len(page.get_images())} image(s). "
                    f"OCR required for full compliance scan.]"
                )

            # Detect encoding metadata for the encoding compliance node
            if text.strip():
                raw_bytes = text.encode("utf-8", errors="replace")
                detected = chardet.detect(raw_bytes)
            else:
                detected = {"encoding": "unknown", "confidence": 0}

            pages_text.append({
                "page_num": page_num + 1,
                "text": text,
                "char_count": len(text),
                "detected_encoding": detected.get("encoding", "unknown"),
                "encoding_confidence": detected.get("confidence", 0.0),
                "image_count": len(page.get_images()),
            })

        doc.close()
        total = len(pages_text)

    except Exception as e:
        errors.append(f"PDF ingestion error: {str(e)}")
        total = 0
```

```
return {
  "total_pages": total,
  "pages_text": pages_text,
  "errors": errors,
}
```

8. AI Compliance Nodes

8.1 config/prompts/pii_prompt.txt

You are an expert PII (Personally Identifiable Information) detection specialist with 20 years of experience in GDPR, DPDP Act, and

Analyse the provided text and identify ALL PII instances. Detect these categories:

- EMAIL: email addresses (e.g., user@domain.com)
- PHONE: phone numbers in any format (+91, (xxx), xxx-xxxx, Indian mobile numbers)
- SSN_AADHAAR: Social Security Numbers, Aadhaar numbers (12-digit), PAN cards, passport numbers
- CREDIT_CARD: credit/debit card numbers (16-digit patterns)
- ADDRESS: physical street addresses, postal codes combined with locations
- NAME_COMBO: full name combined with any other identifier
- BANK_ACCOUNT: bank account numbers, IBAN, SWIFT codes
- DOB: dates of birth
- IP_ADDRESS: IP addresses that could identify individuals
- MEDICAL_ID: patient IDs, medical record numbers

Return ONLY valid JSON in this EXACT schema (no markdown, no explanation):

```
{
  "has_pii": true,
  "findings": [
    {
      "category": "EMAIL",
      "value": "john@example.com",
      "context": "Contact John at john@example.com for",
      "confidence": 0.99,
      "risk_level": "high"
    }
  ],
  "page_risk": "low",
  "recommendation": "Redact email address before sharing"
}
```

Risk levels: low (0.5-0.69), medium (0.7-0.84), high (0.85-0.94), critical (0.95+)

If no PII found: return {"has_pii": false, "findings": [], "page_risk": "low", "recommendation": "No action required"}

8.2 config/prompts/confidential_prompt.txt

You are an expert information security auditor specialising in ISO 27001, SOC 2, and corporate data classification.

Analyse the provided text for confidential or sensitive business information. Detect:

- API_KEY: API keys, access tokens, bearer tokens, JWT secrets
- PASSWORD: passwords, passphrases, secret keys
- AWS_CREDENTIAL: AWS access key IDs, secret access keys
- TRADE_SECRET: proprietary formulas, processes, unreleased product details
- MERGER_ACQUISITION: M&A activity, acquisition targets, deal values
- SALARY_DATA: individual or aggregate compensation data
- INTERNAL_CODENAME: internal project names, operation names
- FINANCIAL_FORECAST: unreleased revenue projections, earnings data
- CUSTOMER_LIST: proprietary customer databases or lists
- SOURCE_CODE: proprietary algorithm logic or source code snippets
- CUSTOM_KEYWORD: any custom keywords flagged in the rules

Return ONLY valid JSON (no markdown):

```
{
  "has_confidential": true,
  "findings": [
    {
      "category": "API_KEY",
      "value": "sk-prod-xxxx...xxxx",
      "context": "surrounding 80 chars",
      "confidence": 0.98,
      "risk_level": "critical"
    }
  ],
  "page_risk": "critical",
  "recommendation": "Immediately revoke exposed API key"
}
```

If nothing found: {"has_confidential": false, "findings": [], "page_risk": "low", "recommendation": "No action required"}

8.3 config/prompts/abuse_prompt.txt

Analyse the provided text for abusive, unlawful, or harmful content. Detect:

- Return ONLY valid JSON (no markdown):

Zero tolerance: `child_safety` and `terrorism` always return "critical".

```
# pipeline/nodes/pii_detector.py
"""
PII Detection Node – detects Personally Identifiable Information using
a combination of rule-based regex and AI classification.
"""

import re
import json
from pathlib import Path
from pipeline.state import PipelineState
from config.ai_provider import call_ai, parse_json_response

# Load system prompt from file
PROMPT_FILE = Path(__file__).parent.parent.parent / "config" / "prompts" / "pii_prompt.txt"
PII_SYSTEM_PROMPT = PROMPT_FILE.read_text() if PROMPT_FILE.exists() else "Detect PII and return JSON."

# Regex pre-filters for common PII patterns (faster than AI for obvious cases)
EMAIL_REGEX = re.compile(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b')
PHONE_REGEX = re.compile(r'(\+?\d{1,3}[-.\s]?)?(?\d{3}\s)?[-.\s]?\d{3}[-.\s]?\d{4}')
AADHAAR_REGEX = re.compile(r'\b\d{4}\s\d{4}\s\d{4}\b')
PAN_REGEX = re.compile(r'\b[A-Z]{5}[0-9]{4}[A-Z]\b')
CREDIT_CARD_REGEX = re.compile(r'\b\d{4}[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}\b')

def _regex_precheck(text: str) -> bool:
    """Quick regex check – if any pattern matches, send to AI for confirmation."""
    return bool(
        EMAIL_REGEX.search(text) or
        PHONE_REGEX.search(text) or
        AADHAAR_REGEX.search(text) or
        PAN_REGEX.search(text) or
        CREDIT_CARD_REGEX.search(text)
    )
```

```

def pii_node(state: PipelineState) -> dict:
    """
    Run PII detection on every page.
    Strategy: regex pre-filter → AI analysis → parse JSON result.
    """
    rules = state.get("compliance_rules", {}).get("pii", {})
    if not rules.get("enabled", True):
        return {"pii_results": []}

    min_confidence = rules.get("min_confidence", 0.75)
    page_results = []

    for page_data in state["pages_text"]:
        text = page_data["text"].strip()

        # Skip empty pages
        if not text or text.startswith("[PAGE]"):
            page_results.append({
                "page_num": page_data["page_num"],
                "pii_flags": [],
                "pii_risk": "low",
            })
            continue

        # Truncate to 4000 chars to stay within token limits
        text_for_ai = text[:4000]
        if len(text) > 4000:
            text_for_ai += "\n[TEXT TRUNCATED]"

        try:
            raw_response = call_ai(
                system_prompt=PII_SYSTEM_PROMPT,
                user_message=f"Analyse page {page_data['page_num']}: \n\n{text_for_ai}",
                max_tokens=1024
            )
            result = parse_json_response(raw_response)

            # Filter by confidence threshold
            findings = [
                f for f in result.get("findings", [])
                if f.get("confidence", 0) >= min_confidence
            ]

            page_results.append({
                "page_num": page_data["page_num"],
                "pii_flags": findings,
                "pii_risk": result.get("page_risk", "low"),
            })

        except Exception as e:
            page_results.append({
                "page_num": page_data["page_num"],
                "pii_flags": [],
                "pii_risk": "unknown",
                "pii_error": str(e),
            })

    return {"pii_results": page_results}

```

8.5 pipeline/nodes/confidentiality.py

```

# pipeline/nodes/confidentiality.py
"""

```

```

"""
Confidentiality Node – detects API keys, trade secrets, and sensitive business data.
"""

import re
from pathlib import Path
from pipeline.state import PipelineState
from config.ai_provider import call_ai, parse_json_response

PROMPT_FILE = Path(__file__).parent.parent.parent / "config" / "prompts" / "confidential_prompt.txt"
BASE_PROMPT = PROMPT_FILE.read_text() if PROMPT_FILE.exists() else "Detect confidential data and return JSON."

# Strong regex patterns for credentials
API_KEY_PATTERNS = [
    re.compile(r'\bsk-[a-zA-Z0-9]{32,}\b'),          # OpenAI / Anthropic keys
    re.compile(r'\bAKIA[A-Z0-9]{16}\b'),            # AWS Access Key IDs
    re.compile(r'\bghp_[a-zA-Z0-9]{36}\b'),          # GitHub Personal tokens
    re.compile(r'\bgsk_[a-zA-Z0-9]{32,}\b'),          # Groq API keys
    re.compile(r'password\s*[=:]s*\S+', re.I),        # password = value patterns
    re.compile(r'secret\s*[=:]s*\S+', re.I),          # secret = value patterns
]

def _build_prompt_with_keywords(rules: dict) -> str:
    """Inject custom keywords from rules into the system prompt."""
    custom_keywords = rules.get("confidentiality", {}).get("custom_keywords", [])
    if not custom_keywords:
        return BASE_PROMPT

    keywords_section = "\n\nAdditional custom keywords to flag as CUSTOM_KEYWORD:\n"
    keywords_section += "\n".join(f"- {kw}" for kw in custom_keywords if kw.strip())
    return BASE_PROMPT + keywords_section

def confidentiality_node(state: PipelineState) -> dict:
    """Run confidentiality detection on every page."""
    rules = state.get("compliance_rules", {})
    conf_rules = rules.get("confidentiality", {})

    if not conf_rules.get("enabled", True):
        return {"confidential_results": []}

    system_prompt = _build_prompt_with_keywords(rules)
    min_confidence = conf_rules.get("min_confidence", 0.70)
    page_results = []

    for page_data in state["pages_text"]:
        text = page_data["text"].strip()

        if not text or text.startswith("[PAGE]"):
            page_results.append({
                "page_num": page_data["page_num"],
                "confidential_flags": [],
                "confidential_risk": "low",
            })
            continue

        # Regex pre-screen for obvious API keys
        regex_hits = []
        for pattern in API_KEY_PATTERNS:
            matches = pattern.findall(text)
            if matches:
                regex_hits.append({"type": "API_KEY_REGEX", "count": len(matches)})

        text_for_ai = text[:4000]

    try:

```

```

raw_response = call_ai(
    system_prompt=system_prompt,
    user_message=f"Analyse page {page_data['page_num']}: \n\n{text_for_ai}",
    max_tokens=1024
)
result = parse_json_response(raw_response)

findings = [
    f for f in result.get("findings", [])
    if f.get("confidence", 0) >= min_confidence
]

page_results.append({
    "page_num": page_data["page_num"],
    "confidential_flags": findings,
    "confidential_risk": result.get("page_risk", "low"),
    "regex_hits": regex_hits,
})

except Exception as e:
    page_results.append({
        "page_num": page_data["page_num"],
        "confidential_flags": regex_hits,
        "confidential_risk": "unknown",
        "confidential_error": str(e),
    })

return {"confidential_results": page_results}

```

8.6 pipeline/nodes/encoding_guard.py

```

# pipeline/nodes/encoding_guard.py
"""
Encoding Guard Node – hybrid rule-based + AI.
Checks UTF-8 consistency, language detection, and non-ASCII characters.
No AI calls needed for this node (pure rule-based).
"""

from langdetect import detect, LangDetectException
from pipeline.state import PipelineState

def encoding_node(state: PipelineState) -> dict:
    """
    Rule-based encoding and language detection.
    Flags: non-ASCII chars, low encoding confidence, non-English text.
    """
    rules = state.get("compliance_rules", {}).get("encoding", {})

    if not rules.get("enabled", True):
        return {"encoding_results": []}

    allowed_languages = rules.get("allowed_languages", ["en"])
    non_ascii_threshold = rules.get("non_ascii_threshold", 5)
    min_enc_confidence = rules.get("min_encoding_confidence", 0.85)
    encoding_results = []

    for page_data in state["pages_text"]:
        flags = []
        text = page_data["text"]

        # Skip image-only pages
        if text.startswith("[PAGE]") and "Image-only" in text:
            encoding_results.append({
                "page_num": page_data["page_num"],

```



```

        "encoding_flags": [{"type": "IMAGE_ONLY_PAGE", "severity": "info"}],
        "encoding_risk": "low",
    })
    continue

# Check 1: Non-ASCII characters
# Allow: typographic quotes, em-dash, ellipsis, currency symbols
allowed_non_ascii = set('""'""'---...€£¥°@®™αβγδ")
non_ascii = [
    (i, ch) for i, ch in enumerate(text)
    if ord(ch) > 127 and ch not in allowed_non_ascii
]

if len(non_ascii) > non_ascii_threshold:
    samples = [ch for _, ch in non_ascii[:10]]
    flags.append({
        "type": "NON_ASCII_CHARS",
        "count": len(non_ascii),
        "samples": samples,
        "severity": "high" if len(non_ascii) > 50 else "medium"
    })

# Check 2: Encoding confidence from chardet
if page_data["encoding_confidence"] < min_enc_confidence:
    flags.append({
        "type": "LOW_ENCODING_CONFIDENCE",
        "detected": page_data["detected_encoding"],
        "confidence": round(page_data["encoding_confidence"], 2),
        "severity": "high"
    })

# Check 3: Language detection (only on meaningful text)
clean_text = text.strip()
if len(clean_text) > 100:
    try:
        detected_lang = detect(clean_text)
        if detected_lang not in allowed_languages:
            flags.append({
                "type": "NON_ENGLISH_LANGUAGE",
                "detected_lang": detected_lang,
                "severity": "medium",
                "note": f"Expected: {allowed_languages}"
            })
    except LangDetectException:
        flags.append({
            "type": "LANGUAGE_UNDETECTABLE",
            "severity": "low",
            "note": "Text too short or mixed for reliable detection"
        })

# Calculate overall encoding risk
severities = [f["severity"] for f in flags]
if "high" in severities:
    risk = "high"
elif "medium" in severities:
    risk = "medium"
elif flags:
    risk = "low"
else:
    risk = "low"

encoding_results.append({
    "page_num": page_data["page_num"],
    "encoding_flags": flags,
    "encoding_risk": risk,
})

```

```
//  
  
return {"encoding_results": encoding_results}
```

8.7 pipeline/nodes/abuse_detector.py

```
# pipeline/nodes/abuse_detector.py  
"""  
Abuse Detection Node – detects hate speech, harassment, and unlawful content using AI.  
"""  
  
from pathlib import Path  
from pipeline.state import PipelineState  
from config.ai_provider import call_ai, parse_json_response  
  
PROMPT_FILE = Path(__file__).parent.parent.parent / "config" / "prompts" / "abuse_prompt.txt"  
ABUSE_SYSTEM_PROMPT = PROMPT_FILE.read_text() if PROMPT_FILE.exists() else "Detect abusive content and return JSON."  
  
def abuse_node(state: PipelineState) -> dict:  
    """Run abuse and unlawful content detection on every page."""  
    rules = state.get("compliance_rules", {}).get("abuse", {})  
  
    if not rules.get("enabled", True):  
        return {"abuse_results": []}  
  
    zero_tolerance = rules.get("zero_tolerance_categories", ["child_safety", "terrorism"])  
    page_results = []  
  
    for page_data in state["pages_text"]:  
        text = page_data["text"].strip()  
  
        if not text or text.startswith("[PAGE]"):   
            page_results.append({  
                "page_num": page_data["page_num"],  
                "abuse_flags": [],  
                "abuse_risk": "low",  
            })  
            continue  
  
        text_for_ai = text[:4000]  
  
        try:  
            raw_response = call_ai(  
                system_prompt=ABUSE_SYSTEM_PROMPT,  
                user_message=f"Analyse page {page_data['page_num']}: \n\n{text_for_ai}",  
                max_tokens=512  
            )  
            result = parse_json_response(raw_response)  
  
            findings = result.get("findings", [])  
  
            # Apply zero-tolerance: override risk to critical  
            for finding in findings:  
                category_lower = finding.get("category", "").lower()  
                if any(zt in category_lower for zt in zero_tolerance):  
                    finding["risk_level"] = "critical"  
  
            # Determine page risk  
            page_risk = result.get("page_risk", "low")  
            if any(f.get("risk_level") == "critical" for f in findings):  
                page_risk = "critical"  
  
            page_results.append({  
                "page_num": page_data["page_num"],  
                "abuse_flags": findings,
```

```

        "abuse_risk": page_risk,
    })

except Exception as e:
    page_results.append({
        "page_num": page_data["page_num"],
        "abuse_flags": [],
        "abuse_risk": "unknown",
        "abuse_error": str(e),
    })

return {"abuse_results": page_results}

```

8.8 pipeline/nodes/aggregator.py

```

# pipeline/nodes/aggregator.py
"""
Aggregator Node – combines results from all 4 compliance nodes into unified page results.
Computes overall risk per page and summary statistics.
"""

from pipeline.state import PipelineState

RISK_RANK = {"low": 0, "medium": 1, "high": 2, "critical": 3, "unknown": 1}

def _highest_risk(*risks: str) -> str:
    """Return the highest risk level from multiple inputs."""
    return max(risks, key=lambda r: RISK_RANK.get(r, 0))

def _count_flags(results: list, page_num: int, flag_key: str) -> list:
    """Get flags for a specific page from results list."""
    for r in results:
        if r.get("page_num") == page_num:
            return r.get(flag_key, [])
    return []

def aggregator_node(state: PipelineState) -> dict:
    """Merge all compliance results into unified page results and summary stats."""
    pii_results = state.get("pii_results", [])
    conf_results = state.get("confidential_results", [])
    enc_results = state.get("encoding_results", [])
    abuse_results = state.get("abuse_results", [])

    page_results = []
    total_issues = {"pii": 0, "confidential": 0, "encoding": 0, "abuse": 0}
    risk_counts = {"low": 0, "medium": 0, "high": 0, "critical": 0}

    for page_data in state.get("pages_text", []):
        page_num = page_data["page_num"]

        # Get results per check
        pii_page = next((r for r in pii_results if r["page_num"] == page_num), {})
        conf_page = next((r for r in conf_results if r["page_num"] == page_num), {})
        enc_page = next((r for r in enc_results if r["page_num"] == page_num), {})
        abuse_page = next((r for r in abuse_results if r["page_num"] == page_num), {})

        pii_flags = pii_page.get("pii_flags", [])
        conf_flags = conf_page.get("confidential_flags", [])
        enc_flags = enc_page.get("encoding_flags", [])
        abuse_flags = abuse_page.get("abuse_flags", [])

        # Tally totals
        total_issues["pii"] += len(pii_flags)
        total_issues["confidential"] += len(conf_flags)
        total_issues["encoding"] += len(enc_flags)
        total_issues["abuse"] += len(abuse_flags)

        # Calculate risk
        risk = _highest_risk(
            "pii" if len(pii_flags) > 0 else None,
            "confidential" if len(conf_flags) > 0 else None,
            "encoding" if len(enc_flags) > 0 else None,
            "abuse" if len(abuse_flags) > 0 else None
        )

        page_results.append({
            "page_num": page_num,
            "pii_flags": pii_flags,
            "confidential_flags": conf_flags,
            "encoding_flags": enc_flags,
            "abuse_flags": abuse_flags,
            "risk": risk
        })

    return {
        "page_results": page_results,
        "total_issues": total_issues,
        "risk_counts": risk_counts
    }

```

```

total_issues["pii"] += len(pii_flags)
total_issues["confidential"] += len(conf_flags)
total_issues["encoding"] += len(enc_flags)
total_issues["abuse"] += len(abuse_flags)

# Calculate overall page risk
overall_risk = _highest_risk(
    pii_page.get("pii_risk", "low"),
    conf_page.get("confidential_risk", "low"),
    enc_page.get("encoding_risk", "low"),
    abuse_page.get("abuse_risk", "low"),
)

risk_counts[overall_risk] = risk_counts.get(overall_risk, 0) + 1

page_results.append({
    "page_num": page_num,
    "text_preview": page_data["text"][:200],
    "char_count": page_data["char_count"],
    "pii_flags": pii_flags,
    "confidential_flags": conf_flags,
    "encoding_flags": enc_flags,
    "abuse_flags": abuse_flags,
    "pii_risk": pii_page.get("pii_risk", "low"),
    "confidential_risk": conf_page.get("confidential_risk", "low"),
    "encoding_risk": enc_page.get("encoding_risk", "low"),
    "abuse_risk": abuse_page.get("abuse_risk", "low"),
    "overall_risk": overall_risk,
    "total_flags": len(pii_flags) + len(conf_flags) + len(enc_flags) + len(abuse_flags),
})

summary = {
    "total_pages": state.get("total_pages", 0),
    "total_issues": total_issues,
    "risk_counts": risk_counts,
    "total_flags": sum(total_issues.values()),
    "highest_risk": _highest_risk(*[p["overall_risk"] for p in page_results]) if page_results else "low",
    "pages_with_issues": len([p for p in page_results if p["total_flags"] > 0]),
    "clean_pages": len([p for p in page_results if p["total_flags"] == 0]),
}

return {
    "page_results": page_results,
    "summary": summary,
    "processing_complete": True,
}

```

9. LangGraph Orchestration

9.1 pipeline/graph.py

```

# pipeline/graph.py
"""
LangGraph pipeline builder – creates the compliance scanning DAG.
Architecture: ingest → 4 parallel compliance nodes → aggregator → report
"""

from langgraph.graph import StateGraph, END
from pipeline.state import PipelineState
from pipeline.nodes.ingest import ingest_node
from pipeline.nodes.pii_detector import pii_node
from pipeline.nodes.confidentiality import confidentiality_node
from pipeline.nodes.encoding_guard import encoding_node
from pipeline.nodes.abuse_detector import abuse_node

```

```

from pipeline.nodes.abuse_detector import abuse_node
from pipeline.nodes.aggregator import aggregator_node
from pipeline.nodes.report_builder import report_node

def build_pipeline() -> StateGraph:
    """
    Build and compile the LangGraph compliance pipeline.

    Graph structure:
        ingest -> [pii_check, confidentiality, encoding_check, abuse_check] (parallel)
                -> aggregate -> build_report -> END
    """
    graph = StateGraph(PipelineState)

    # Register all nodes
    graph.add_node("ingest", ingest_node)
    graph.add_node("pii_check", pii_node)
    graph.add_node("confidentiality", confidentiality_node)
    graph.add_node("encoding_check", encoding_node)
    graph.add_node("abuse_check", abuse_node)
    graph.add_node("aggregate", aggregator_node)
    graph.add_node("build_report", report_node)

    # Entry point
    graph.set_entry_point("ingest")

    # After ingestion, run all 4 compliance checks in parallel
    graph.add_edge("ingest", "pii_check")
    graph.add_edge("ingest", "confidentiality")
    graph.add_edge("ingest", "encoding_check")
    graph.add_edge("ingest", "abuse_check")

    # All compliance nodes feed into aggregator
    graph.add_edge("pii_check", "aggregate")
    graph.add_edge("confidentiality", "aggregate")
    graph.add_edge("encoding_check", "aggregate")
    graph.add_edge("abuse_check", "aggregate")

    # After aggregation, build report
    graph.add_conditional_edges(
        "aggregate",
        lambda s: "build_report" if s.get("processing_complete") else "aggregate",
        {"build_report": "build_report", "aggregate": "aggregate"}
    )

    graph.add_edge("build_report", END)

    return graph.compile()

def run_pipeline(pdf_path: str, pdf_name: str, upload_id: str, compliance_rules: dict) -> dict:
    """
    Convenience function to run the full compliance pipeline.

    Args:
        pdf_path: Path to uploaded PDF file
        pdf_name: Original filename for display
        upload_id: Unique scan identifier
        compliance_rules: Rules dict from rules.json

    Returns:
        Final pipeline state with all results
    """
    pipeline = build_pipeline()

```

```

initial_state = {
    "pdf_path": pdf_path,
    "pdf_name": pdf_name,
    "upload_id": upload_id,
    "total_pages": 0,
    "pages_text": [],
    "pii_results": [],
    "confidential_results": [],
    "encoding_results": [],
    "abuse_results": [],
    "page_results": [],
    "summary": {},
    "compliance_rules": compliance_rules,
    "report_path": None,
    "processing_complete": False,
    "errors": [],
}

return pipeline.invoke(initial_state)

```

10. Database & Storage Layer

10.1 storage/database.py

```

# storage/database.py
"""
SQLite-based persistence for scan results.
Each upload gets a UUID; results stored as JSON blobs.
"""

import sqlite3
import json
import os
from datetime import datetime
from pathlib import Path

DB_PATH = Path(__file__).parent / "compliance.db"

def get_connection():
    """Get a SQLite connection with WAL mode for concurrent reads."""
    conn = sqlite3.connect(str(DB_PATH))
    conn.execute("PRAGMA journal_mode=WAL")
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    """Create database tables if they don't exist."""
    with get_connection() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS scans (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                upload_id TEXT UNIQUE NOT NULL,
                pdf_name TEXT NOT NULL,
                scanned_at TEXT NOT NULL,
                total_pages INTEGER DEFAULT 0,
                total_flags INTEGER DEFAULT 0,
                highest_risk TEXT DEFAULT 'low',
                status TEXT DEFAULT 'completed',
                result_json TEXT,
                report_path TEXT
            )

```

```

    """
    conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_upload_id ON scans(upload_id)
    """)
    conn.commit()

```

```

def save_result(upload_id: str, pdf_name: str, result: dict) -> None:

```

```

    """Save scan result to database."""

```

```

    init_db()

```

```

    summary = result.get("summary", {})

```

```

    with get_connection() as conn:

```

```

        conn.execute("""

```

```

            INSERT OR REPLACE INTO scans

```

```

            (upload_id, pdf_name, scanned_at, total_pages, total_flags,
             highest_risk, status, result_json, report_path)

```

```

            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)

```

```

        """, (

```

```

            upload_id,

```

```

            pdf_name,

```

```

            datetime.now().isoformat(),

```

```

            result.get("total_pages", 0),

```

```

            summary.get("total_flags", 0),

```

```

            summary.get("highest_risk", "low"),

```

```

            "completed",

```

```

            json.dumps(result), # Store full result as JSON

```

```

            result.get("report_path"),

```

```

        ))

```

```

        conn.commit()

```

```

def get_result(upload_id: str) -> dict | None:

```

```

    """Retrieve scan result by upload ID."""

```

```

    init_db()

```

```

    with get_connection() as conn:

```

```

        row = conn.execute(

```

```

            "SELECT * FROM scans WHERE upload_id = ?", (upload_id,)

```

```

        ).fetchone()

```

```

    if not row:

```

```

        return None

```

```

    result = dict(row)

```

```

    if result.get("result_json"):

```

```

        result["data"] = json.loads(result["result_json"])

```

```

    return result

```

```

def get_all_scans() -> list:

```

```

    """Get all scan records (metadata only, not full JSON)."""

```

```

    init_db()

```

```

    with get_connection() as conn:

```

```

        rows = conn.execute("""

```

```

            SELECT upload_id, pdf_name, scanned_at, total_pages,

```

```

                   total_flags, highest_risk, status, report_path

```

```

            FROM scans

```

```

            ORDER BY scanned_at DESC

```

```

            LIMIT 50

```

```

        """).fetchall()

```

```

    return [dict(row) for row in rows]

```

```

def delete_scan(upload_id: str) -> bool:

```

```

    """Delete a scan record."""

```

```

    with get_connection() as conn:

```

```
with get_connection() as conn:
    cursor = conn.execute("DELETE FROM scans WHERE upload_id = ?", (upload_id,))
    conn.commit()
    return cursor.rowcount > 0
```

11. Report Generation Engine

11.1 pipeline/nodes/report_builder.py

```
# pipeline/nodes/report_builder.py
"""
Report Builder Node – generates professional PDF compliance reports using ReportLab.
Includes executive summary, risk heatmap table, and per-finding details.
"""

import os
from datetime import datetime
from pathlib import Path
from pipeline.state import PipelineState

# ReportLab imports
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Table, TableStyle,
    Spacer, HRFlowable, PageBreak
)
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import cm
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_LEFT

# Color palette
COLOR_CRITICAL = colors.HexColor("#C0392B")
COLOR_HIGH = colors.HexColor("#E67E22")
COLOR_MEDIUM = colors.HexColor("#F39C12")
COLOR_LOW = colors.HexColor("#27AE60")
COLOR_HEADER = colors.HexColor("#2C3E50")
COLOR_LIGHT_GRAY = colors.HexColor("#ECF0F1")
COLOR_WHITE = colors.white

RISK_COLORS = {
    "critical": COLOR_CRITICAL,
    "high": COLOR_HIGH,
    "medium": COLOR_MEDIUM,
    "low": COLOR_LOW,
    "unknown": colors.gray,
}

def _risk_badge_color(risk: str) -> colors.Color:
    return RISK_COLORS.get(risk.lower(), colors.gray)

def report_node(state: PipelineState) -> dict:
    """Generate a PDF compliance report from pipeline results."""
    os.makedirs("reports", exist_ok=True)

    # Sanitize filename
    safe_name = "".join(c for c in state["pdf_name"] if c.isalnum() or c in "._- ")
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    report_path = f"reports/{safe_name}_{state['upload_id']}_{timestamp}_compliance.pdf"

    doc = SimpleDocTemplate(
```



```

        report_path,
        pagesize=A4,
        rightMargin=2*cm, leftMargin=2*cm,
        topMargin=2*cm, bottomMargin=2*cm
    )

    styles = getSampleStyleSheet()
    story = []

    # Custom styles
    title_style = ParagraphStyle(
        "CustomTitle",
        parent=styles["Title"],
        fontSize=20,
        textColor=COLOR_HEADER,
        spaceAfter=6,
        alignment=TA_CENTER,
    )
    subtitle_style = ParagraphStyle(
        "Subtitle",
        parent=styles["Normal"],
        fontSize=10,
        textColor=colors.gray,
        alignment=TA_CENTER,
        spaceAfter=20,
    )
    section_heading = ParagraphStyle(
        "SectionHeading",
        parent=styles["Heading2"],
        fontSize=13,
        textColor=COLOR_HEADER,
        spaceBefore=12,
        spaceAfter=6,
        borderPad=4,
    )
    body_style = ParagraphStyle(
        "Body",
        parent=styles["Normal"],
        fontSize=9,
        spaceAfter=4,
        leading=14,
    )

    # — TITLE BLOCK —————
    story.append(Paragraph("PDF Compliance Report", title_style))
    story.append(Paragraph(
        f"Document: <b>{state['pdf_name']}</b> | "
        f"Scan ID: {state['upload_id']} | "
        f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
        subtitle_style
    ))
    story.append(HRFlowable(width="100%", thickness=2, color=COLOR_HEADER))
    story.append(Spacer(1, 0.4*cm))

    # — EXECUTIVE SUMMARY —————
    summary = state.get("summary", {})
    story.append(Paragraph("Executive Summary", section_heading))

    summary_data = [
        ["Metric", "Value"],
        ["Total Pages Scanned", str(summary.get("total_pages", 0))],
        ["Total Compliance Flags", str(summary.get("total_flags", 0))],
        ["Pages with Issues", str(summary.get("pages_with_issues", 0))],
        ["Clean Pages", str(summary.get("clean_pages", 0))],
        ["Highest Risk Level", summary.get("highest_risk", "low").upper()],
    ]

```

```

["PII Violations", str(summary.get("total_issues", {}).get("pii", 0))],
["Confidentiality Issues", str(summary.get("total_issues", {}).get("confidential", 0))],
["Encoding Issues", str(summary.get("total_issues", {}).get("encoding", 0))],
["Abuse/Unlawful Content", str(summary.get("total_issues", {}).get("abuse", 0))],
]

summary_table = Table(summary_data, colWidths=[8*cm, 8*cm])
highest_risk = summary.get("highest_risk", "low")
risk_color = _risk_badge_color(highest_risk)

summary_table.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), COLOR_HEADER),
    ("TEXTCOLOR", (0, 0), (-1, 0), COLOR_WHITE),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
    ("FONTSIZE", (0, 0), (-1, 0), 10),
    ("BACKGROUND", (0, 5), (-1, 5), risk_color),
    ("TEXTCOLOR", (0, 5), (-1, 5), COLOR_WHITE),
    ("FONTNAME", (0, 5), (-1, 5), "Helvetica-Bold"),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1), [COLOR_LIGHT_GRAY, COLOR_WHITE]),
    ("GRID", (0, 0), (-1, -1), 0.5, colors.gray),
    ("FONTSIZE", (0, 1), (-1, -1), 9),
    ("PADDING", (0, 0), (-1, -1), 6),
]))
story.append(summary_table)
story.append(Spacer(1, 0.5*cm))

# — RISK HEATMAP TABLE —————
story.append(Paragraph("Page-by-Page Risk Heatmap", section_heading))

heatmap_header = ["Page", "PII", "Confidential", "Encoding", "Abuse", "Flags", "Risk"]
heatmap_data = [heatmap_header]

for pr in state.get("page_results", []):
    row = [
        str(pr["page_num"]),
        str(len(pr.get("pii_flags", []))),
        str(len(pr.get("confidential_flags", []))),
        str(len(pr.get("encoding_flags", []))),
        str(len(pr.get("abuse_flags", []))),
        str(pr.get("total_flags", 0)),
        pr.get("overall_risk", "low").upper(),
    ]
    heatmap_data.append(row)

heatmap_table = Table(
    heatmap_data,
    colWidths=[1.5*cm, 2*cm, 2.5*cm, 2*cm, 1.8*cm, 1.5*cm, 2*cm]
)

heatmap_style = [
    ("BACKGROUND", (0, 0), (-1, 0), COLOR_HEADER),
    ("TEXTCOLOR", (0, 0), (-1, 0), COLOR_WHITE),
    ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
    ("FONTSIZE", (0, 0), (-1, -1), 8),
    ("GRID", (0, 0), (-1, -1), 0.3, colors.gray),
    ("ALIGN", (0, 0), (-1, -1), "CENTER"),
    ("PADDING", (0, 0), (-1, -1), 4),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1), [COLOR_LIGHT_GRAY, COLOR_WHITE]),
]

# Color-code risk column
for i, pr in enumerate(state.get("page_results", []), start=1):
    risk = pr.get("overall_risk", "low")
    if risk in ["critical", "high"]:
        heatmap_style.append((
            "BACKGROUND", (6, i), (6, i), risk_badge_color(risk)

```

```

        BACKGROUND", (0, 1), (0, 1), _risk_badge_color(pr.get("overall_risk", "low"))
    ))
    heatmap_style.append(("TEXTCOLOR", (6, i), (6, i), COLOR_WHITE))

heatmap_table.setStyle(TableStyle(heatmap_style))
story.append(heatmap_table)
story.append(Spacer(1, 0.5*cm))

# — DETAILED FINDINGS —————
story.append(PageBreak())
story.append(Paragraph("Detailed Findings", section_heading))

for pr in state.get("page_results", []):
    all_flags = (
        [(f, "PII") for f in pr.get("pii_flags", [])] +
        [(f, "CONFIDENTIAL") for f in pr.get("confidential_flags", [])] +
        [(f, "ENCODING") for f in pr.get("encoding_flags", [])] +
        [(f, "ABUSE") for f in pr.get("abuse_flags", [])]
    )

    if not all_flags:
        continue

    story.append(Paragraph(
        f"Page {pr['page_num']} – Risk: {pr['overall_risk'].upper()} "
        f"({len(all_flags)} flag(s))",
        ParagraphStyle("PageHeader", parent=body_style, fontName="Helvetica-Bold",
            textColor=_risk_badge_color(pr.get("overall_risk", "low")))
    ))

    finding_data = [{"Check Type", "Category", "Confidence", "Risk", "Recommendation"}]
    for flag, check_type in all_flags:
        finding_data.append([
            check_type,
            flag.get("type", flag.get("category", "UNKNOWN")),
            f"{flag.get('confidence', 1.0):.0%}" if isinstance(flag.get('confidence'), float) else "N/A",
            flag.get("risk_level", flag.get("severity", "medium")).upper(),
            (flag.get("context", "See document")[:60] + "...") if len(flag.get("context", "")) > 60
            else flag.get("context", flag.get("note", "Review required")),
        ])

    finding_table = Table(finding_data, colwidths=[2.5*cm, 3*cm, 2*cm, 1.8*cm, 6.5*cm])
    finding_table.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), COLOR_HEADER),
        ("TEXTCOLOR", (0, 0), (-1, 0), COLOR_WHITE),
        ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
        ("FONTSIZE", (0, 0), (-1, -1), 7.5),
        ("GRID", (0, 0), (-1, -1), 0.3, colors.lightgrey),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [COLOR_LIGHT_GRAY, COLOR_WHITE]),
        ("PADDING", (0, 0), (-1, -1), 4),
        ("WORDWRAP", (0, 0), (-1, -1), True),
    ]))
    story.append(finding_table)
    story.append(Spacer(1, 0.3*cm))

doc.build(story)

return {
    "report_path": report_path,
    "processing_complete": True,
}

```

12.1 app/main.py (Entry Point)

```
# app/main.py
"""
Streamlit entry point – configures the app and renders the main navigation.
"""

import streamlit as st
from pathlib import Path

st.set_page_config(
    page_title="PDF Compliance Scanner",
    page_icon="📄",
    layout="wide",
    initial_sidebar_state="expanded",
)

# Custom CSS for better styling
st.markdown("""
<style>
    .main-header {
        background: linear-gradient(135deg, #1e3a5f 0%, #2c3e50 100%);
        padding: 20px;
        border-radius: 10px;
        margin-bottom: 20px;
        color: white;
        text-align: center;
    }
    .risk-critical { color: #C0392B; font-weight: bold; }
    .risk-high { color: #E67E22; font-weight: bold; }
    .risk-medium { color: #F39C12; font-weight: bold; }
    .risk-low { color: #27AE60; font-weight: bold; }
    .metric-card {
        background: #262730;
        padding: 15px;
        border-radius: 8px;
        border-left: 4px solid #FF4B4B;
        margin-bottom: 10px;
    }
    .stProgress .st-bo { background-color: #FF4B4B; }
</style>
""", unsafe_allow_html=True)

# Sidebar navigation
with st.sidebar:
    st.markdown("## 📄 PDF Compliance Scanner")
    st.markdown("---")
    st.markdown("""
    **Powered by:**
    - 🦙 Groq Llama 3 (Free AI)
    - 🧠 LangGraph Orchestration
    - 📄 PyMuPDF Extraction
    - 📑 ReportLab Reports
    """)
    st.markdown("---")
    st.caption("v1.0 · Compliance Checks: PII · Confidential · Encoding · Abuse")

# Main page content
st.markdown("""
<div class="main-header">
    <h1>📄 AI-Powered PDF Compliance Scanner</h1>
    <p>Automatically detect PII, confidential data, encoding issues, and abusive content</p>
</div>
""", unsafe_allow_html=True)

col1, col2, col3, col4 = st.columns(4)
```

```

with col1:
    st.info("🔍 **PII Detection**\nEmails, phones, IDs, addresses")
with col2:
    st.warning("🔒 **Confidentiality**\nAPI keys, trade secrets, credentials")
with col3:
    st.success("✅ **Encoding Check**\nUTF-8 validation, language detection")
with col4:
    st.error("⚠️ **Abuse Detection**\nHate speech, harassment, unlawful content")

st.markdown("---")
st.markdown("### 📄 Select a page from the sidebar to get started")
st.markdown("1. **📄 Upload & Scan** – Upload your PDF and run compliance checks")
st.markdown("2. **🔒 Compliance Rules** – Configure detection rules and thresholds")
st.markdown("3. **📊 View Reports** – Browse and download past scan reports")

```

12.2 app/pages/01_upload.py

```

# app/pages/01_upload.py
"""
Upload & Scan page – handles PDF upload and runs the full compliance pipeline.
"""

import streamlit as st
import tempfile
import uuid
import os
import time
from pathlib import Path

from pipeline.graph import run_pipeline
from config.rules import load_rules
from storage.database import save_result

st.set_page_config(page_title="Upload & Scan", page_icon="📄", layout="wide")

st.title("📄 Upload PDF for Compliance Scan")
st.markdown("Upload any PDF document to automatically scan for compliance violations.")

# File uploader
uploaded_file = st.file_uploader(
    "Choose a PDF file",
    type=["pdf"],
    help="Maximum 50MB. Scans for PII, confidential data, encoding issues, and abusive content."
)

if uploaded_file is None:
    st.info("📄 Upload a PDF file to begin scanning")
    st.markdown("""
    **What gets checked:**
    | Check | Examples |
    |-----|-----|
    | PII Detection | Emails, phone numbers, Aadhaar, credit cards, addresses |
    | Confidentiality | API keys, passwords, trade secrets, M&A details |
    | Encoding | Non-UTF-8 content, non-English text, OCR artifacts |
    | Abuse Detection | Hate speech, harassment, threats, illegal content |
    """)
    st.stop()

# Show file info
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("📄 File Name", uploaded_file.name[:30] + ("..." if len(uploaded_file.name) > 30 else ""))
with col2:
    size_kb = uploaded_file.size / 1024
    st.metric("📄 File Size", f"{size_kb:1f} KB")

```

```

        st.metric("📄 File Size", f"{size_kb:.1f} KB",
                  delta=delta_size, delta_color="normal")

with col3:
    st.metric("📄 File Type", "PDF Document")

st.markdown("---")

# Scan controls
col_btn, col_info = st.columns([1, 3])
with col_btn:
    scan_btn = st.button("🔍 Run Compliance Scan", type="primary", use_container_width=True)
with col_info:
    st.caption("Scans use Groq's free AI API. Processing takes ~2-5 seconds per page.")

if scan_btn:
    # Save uploaded file to temp location
    with tempfile.NamedTemporaryFile(suffix=".pdf", delete=False) as tmp:
        tmp.write(uploaded_file.read())
        tmp_path = tmp.name

    upload_id = str(uuid.uuid4())[:8]
    rules = load_rules()

    # Progress tracking
    progress_bar = st.progress(0, text="Starting compliance scan...")
    status_text = st.empty()
    start_time = time.time()

    try:
        # Stage 1: Ingestion
        progress_bar.progress(10, text="📄 Extracting text from PDF...")
        status_text.info("📄 Reading document structure and extracting text...")
        time.sleep(0.5)

        # Stage 2: Running pipeline
        progress_bar.progress(20, text="🔍 Running AI compliance checks...")
        status_text.info("🔍 AI nodes analyzing content (PII, Confidentiality, Encoding, Abuse)...")

        result = run_pipeline(
            pdf_path=tmp_path,
            pdf_name=uploaded_file.name,
            upload_id=upload_id,
            compliance_rules=rules,
        )

        # Stage 3: Saving results
        progress_bar.progress(90, text="💾 Saving results...")
        save_result(upload_id, uploaded_file.name, result)

        progress_bar.progress(100, text="🎉 Scan complete!")
        elapsed = time.time() - start_time
        status_text.success(f"🎉 Scan completed in {elapsed:.1f} seconds – Report ID: `{upload_id}`")

        # Store result in session for display
        st.session_state["latest_result"] = result
        st.session_state["latest_upload_id"] = upload_id

    except Exception as e:
        st.error(f"🚨 Scan failed: {str(e)}")
        st.exception(e)

finally:
    # Always clean up temp file
    try:
        os.unlink(tmp_path)
    except:
        pass

```

```

# Display results if scan succeeded
if "latest_result" in st.session_state:
    result = st.session_state["latest_result"]
    summary = result.get("summary", {})
    st.markdown("---")
    st.subheader("🔍 Scan Results")

    # Summary metrics
    m1, m2, m3, m4, m5 = st.columns(5)
    with m1:
        st.metric("📄 Pages Scanned", summary.get("total_pages", 0))
    with m2:
        st.metric("🚩 Total Flags", summary.get("total_flags", 0))
    with m3:
        risk = summary.get("highest_risk", "low").upper()
        st.metric("⚠️ Highest Risk", risk)
    with m4:
        st.metric("🔒 PII Issues", summary.get("total_issues", {}).get("pii", 0))
    with m5:
        st.metric("🔒 Confidential", summary.get("total_issues", {}).get("confidential", 0))

    # Risk breakdown
    st.markdown("#### Issue Breakdown by Type")
    issues = summary.get("total_issues", {})
    col_a, col_b, col_c, col_d = st.columns(4)
    with col_a:
        st.error(f"🔒 PII: {issues.get('pii', 0)} flags")
    with col_b:
        st.warning(f"🔒 Confidential: {issues.get('confidential', 0)} flags")
    with col_c:
        st.info(f"🔒 Encoding: {issues.get('encoding', 0)} flags")
    with col_d:
        st.error(f"⚠️ Abuse: {issues.get('abuse', 0)} flags")

    # Page-by-page results table
    st.markdown("#### Page-by-Page Risk Summary")
    page_data = []
    for pr in result.get("page_results", []):
        page_data.append({
            "Page": pr["page_num"],
            "PII Flags": len(pr.get("pii_flags", [])),
            "Confidential": len(pr.get("confidential_flags", [])),
            "Encoding": len(pr.get("encoding_flags", [])),
            "Abuse": len(pr.get("abuse_flags", [])),
            "Total Flags": pr.get("total_flags", 0),
            "Overall Risk": pr.get("overall_risk", "low").upper(),
        })

    if page_data:
        st.dataframe(page_data, use_container_width=True)

    # Download report
    report_path = result.get("report_path")
    if report_path and Path(report_path).exists():
        st.markdown("---")
        with open(report_path, "rb") as f:
            st.download_button(
                label="📄 Download Full Compliance Report (PDF)",
                data=f.read(),
                file_name=f"compliance_report_{upload_id}.pdf",
                mime="application/pdf",
                use_container_width=True,
            )

    # Errors
    st.markdown("#### Error Log")
    errors = result.get("errors", [])
    if errors:
        st.error(f"❌ {len(errors)} errors found during scan.")
        st.table(errors)
    else:
        st.success("✅ No errors found during scan.")

```

```

if result.get("errors"):
    with st.expander("⚠ Scan Warnings"):
        for err in result["errors"]:
            st.warning(err)

```

12.3 app/pages/02_rules.py

```

# app/pages/02_rules.py
"""
Compliance Rules Editor – configure detection rules via UI.
Changes take effect on the next scan – no restart needed.
"""

import streamlit as st
from config.rules import load_rules, save_rules

st.set_page_config(page_title="Compliance Rules", page_icon="🛡", layout="wide")

st.title("🛡 Compliance Rules Configuration")
st.markdown("Customize what the scanner detects. Changes are saved instantly and apply to the next scan.")

rules = load_rules()

with st.form("compliance_rules_form"):
    st.subheader("🔍 PII Detection")
    col1, col2 = st.columns(2)
    with col1:
        pii_enabled = st.toggle("Enable PII Detection", value=rules["pii"]["enabled"])
        detect_email = st.checkbox("📧 Email Addresses", value=rules["pii"].get("detect_email", True))
        detect_phone = st.checkbox("📞 Phone Numbers", value=rules["pii"].get("detect_phone", True))
        detect_ssn = st.checkbox("📄 SSN / Aadhaar / PAN", value=rules["pii"].get("detect_ssn_aadhaar", True))
    with col2:
        detect_credit_card = st.checkbox("💳 Credit/Debit Cards", value=rules["pii"].get("detect_credit_card", True))
        detect_address = st.checkbox("🏠 Physical Addresses", value=rules["pii"].get("detect_address", True))
        detect_dob = st.checkbox("📅 Dates of Birth", value=rules["pii"].get("detect_dob", True))
        pii_confidence = st.slider(
            "Minimum PII Confidence Threshold",
            0.5, 1.0,
            value=rules["pii"].get("min_confidence", 0.75),
            step=0.05,
            help="Flags below this confidence level are ignored"
        )

    st.markdown("---")
    st.subheader("🔒 Confidentiality Detection")
    col3, col4 = st.columns(2)
    with col3:
        conf_enabled = st.toggle("Enable Confidentiality Detection", value=rules["confidentiality"]["enabled"])
        detect_api_keys = st.checkbox("🔑 API Keys & Tokens", value=rules["confidentiality"].get("detect_api_keys", True))
        detect_passwords = st.checkbox("🔐 Passwords & Secrets", value=rules["confidentiality"].get("detect_passwords", True))
        detect_trade_secrets = st.checkbox("📜 Trade Secrets", value=rules["confidentiality"].get("detect_trade_secrets", True))
    with col4:
        detect_financial = st.checkbox("💰 Financial Forecasts", value=rules["confidentiality"].get("detect_financial_data", True))
        detect_ma = st.checkbox("📰 M&A Information", value=rules["confidentiality"].get("detect_merger_acquisition", True))
        conf_confidence = st.slider(
            "Minimum Confidentiality Threshold",
            0.5, 1.0,
            value=rules["confidentiality"].get("min_confidence", 0.70),
            step=0.05
        )

    st.markdown("#### Custom Keywords to Flag")
    current_keywords = "\n".join(rules.get("confidentiality", {}).get("custom_keywords", []))
    custom_keywords_text = st.text_area(
        "One keyword per line (case-insensitive)",
        value=current_keywords,

```



```

        height=100,
        help="These words will always be flagged as CUSTOM_KEYWORD in confidentiality checks"
    )

st.markdown("---")
st.subheader("🔒 Encoding & Language")
col5, col6 = st.columns(2)
with col5:
    enc_enabled = st.toggle("Enable Encoding Check", value=rules["encoding"]["enabled"])
    require_utf8 = st.checkbox("Require UTF-8 Encoding", value=rules["encoding"].get("require_utf8", True))
    flag_non_ascii = st.checkbox("Flag Non-ASCII Characters", value=rules["encoding"].get("flag_non_ascii", True))
with col6:
    non_ascii_threshold = st.number_input(
        "Non-ASCII character threshold (flag if count exceeds)",
        min_value=1, max_value=100,
        value=rules["encoding"].get("non_ascii_threshold", 5)
    )
    enc_confidence = st.slider(
        "Min Encoding Detection Confidence",
        0.5, 1.0,
        value=rules["encoding"].get("min_encoding_confidence", 0.85),
        step=0.05
    )

st.markdown("---")
st.subheader("⚠️ Abuse & Unlawful Content")
col7, col8 = st.columns(2)
with col7:
    abuse_enabled = st.toggle("Enable Abuse Detection", value=rules["abuse"]["enabled"])
    detect_hate = st.checkbox("🗨️ Hate Speech", value=rules["abuse"].get("detect_hate_speech", True))
    detect_sexual = st.checkbox("👤 Sexual Content", value=rules["abuse"].get("detect_sexual_content", True))
    detect_violence = st.checkbox("🔪 Violence Incitement", value=rules["abuse"].get("detect_violence_incitement", True))
with col8:
    detect_illegal = st.checkbox("⚖️ Illegal Content", value=rules["abuse"].get("detect_illegal_content", True))
    detect_harassment = st.checkbox("😡 Harassment", value=rules["abuse"].get("detect_harassment", True))
    sensitivity = st.select_slider(
        "AI Detection Sensitivity",
        options=["low", "medium", "high", "very_high"],
        value=rules.get("sensitivity", "high")
    )

st.markdown("---")
submitted = st.form_submit_button("💾 Save Rules", type="primary", use_container_width=True)

if submitted:
    custom_keywords_list = [
        kw.strip() for kw in custom_keywords_text.split("\n")
        if kw.strip()
    ]

    updated_rules = {
        **rules,
        "sensitivity": sensitivity,
        "pii": {
            "enabled": pii_enabled,
            "detect_email": detect_email,
            "detect_phone": detect_phone,
            "detect_ssn_aadhaar": detect_ssn,
            "detect_credit_card": detect_credit_card,
            "detect_address": detect_address,
            "detect_dob": detect_dob,
            "min_confidence": pii_confidence,
            "risk_threshold": "medium",
        },
        "confidentiality": {

```

```

        "enabled": conf_enabled,
        "detect_api_keys": detect_api_keys,
        "detect_passwords": detect_passwords,
        "detect_trade_secrets": detect_trade_secrets,
        "detect_financial_data": detect_financial,
        "detect_merger_acquisition": detect_ma,
        "custom_keywords": custom_keywords_list,
        "min_confidence": conf_confidence,
    },
    "encoding": {
        "enabled": enc_enabled,
        "require_utf8": require_utf8,
        "allowed_languages": ["en"],
        "min_encoding_confidence": enc_confidence,
        "flag_non_ascii": flag_non_ascii,
        "non_ascii_threshold": non_ascii_threshold,
    },
    "abuse": {
        "enabled": abuse_enabled,
        "detect_hate_speech": detect_hate,
        "detect_sexual_content": detect_sexual,
        "detect_violence_incitement": detect_violence,
        "detect_illegal_content": detect_illegal,
        "detect_harassment": detect_harassment,
        "sensitivity_level": sensitivity,
        "zero_tolerance_categories": ["child_safety", "terrorism"],
    },
}

save_rules(updated_rules)
st.success("🎉 Rules saved successfully! They will apply to the next scan.")
st.balloons()

```

12.4 app/pages/03_reports.py

```

# app/pages/03_reports.py
"""
Reports page – browse and download past compliance scan reports.
"""

import streamlit as st
from pathlib import Path
from storage.database import get_all_scans, get_result, delete_scan

st.set_page_config(page_title="View Reports", page_icon="📄", layout="wide")

st.title("📄 Compliance Scan Reports")
st.markdown("Browse and download your past compliance scan reports.")

scans = get_all_scans()

if not scans:
    st.info("No scans yet. Go to **📄 Upload & Scan** to run your first compliance check.")
    st.stop()

st.metric("Total Scans", len(scans))

for scan in scans:
    risk = scan.get("highest_risk", "low")
    risk_emoji = {"critical": "🔴", "high": "🟠", "medium": "🟡", "low": "🟢"}.get(risk, "o")

    with st.expander(
        f"{risk_emoji} {scan['pdf_name']} – {scan['scanned_at'][:16]} | "
        f"Flags: {scan['total_flags']} | Risk: {risk.upper()}"
    ):
        col1, col2, col3, col4 = st.columns(4)
        with col1:
            st.metric("Pages", scan.get("total_pages", 0))
        with col2:
            st.metric("Total Flags", scan.get("total_flags", 0))
        with col3:
            st.metric("Highest Risk", risk.upper())
        with col4:
            st.metric("Scan ID", scan["upload_id"])

    # Download report PDF
    report_path = scan.get("report_path")
    if report_path and Path(report_path).exists():
        with open(report_path, "rb") as f:
            st.download_button(
                label=f"📄 Download Compliance Report",
                data=f.read(),
                file_name=f"compliance_{scan['upload_id']}.pdf",
                mime="application/pdf",
                key=f"dl_{scan['upload_id']}"
            )
    else:
        st.warning("Report PDF not found (may have been deleted or moved)")

    # Full result data
    full = get_result(scan["upload_id"])
    if full and "data" in full:
        with st.expander("📄 View Raw JSON Results"):
            st.json(full["data"].get("summary", {}))

    # Delete button
    if st.button(f"🗑️ Delete Scan", key=f"del_{scan['upload_id']}"):
        delete_scan(scan["upload_id"])
        st.success("Scan deleted.")
        st.rerun()

```

13. Configuration Files

13.1 config/rules.json (default — auto-created if missing)

```
{
  "version": "1.0",
  "sensitivity": "high",
  "pii": {
    "enabled": true,
    "detect_email": true,
    "detect_phone": true,
    "detect_ssn_aadhaar": true,
    "detect_credit_card": true,
    "detect_address": true,
    "detect_dob": true,
    "min_confidence": 0.75,
    "risk_threshold": "medium"
  },
  "confidentiality": {
    "enabled": true,
    "detect_api_keys": true,
    "detect_passwords": true,
    "detect_trade_secrets": true,
    "detect_financial_data": true,
    "detect_merger_acquisition": true,
    "custom_keywords": [
      "PROJECT TITAN",
      "Operation Phoenix",
      "internal use only",
      "confidential",
      "do not distribute"
    ],
    "min_confidence": 0.70
  },
  "encoding": {
    "enabled": true,
    "require_utf8": true,
    "allowed_languages": ["en"],
    "min_encoding_confidence": 0.85,
    "flag_non_ascii": true,
    "non_ascii_threshold": 5
  },
  "abuse": {
    "enabled": true,
    "detect_hate_speech": true,
    "detect_sexual_content": true,
    "detect_violence_incitement": true,
    "detect_illegal_content": true,
    "detect_harassment": true,
    "sensitivity_level": "high",
    "zero_tolerance_categories": ["child_safety", "terrorism"]
  },
  "reporting": {
    "include_text_context": true,
    "context_window_chars": 150,
    "generate_pdf_report": true,
    "generate_json_report": true
  }
}
```

13.2 pipeline/nodes/init.py

```
# pipeline/nodes/__init__.py
from .ingest import ingest_node
from .pii_detector import pii_node
from .confidentiality import confidentiality_node
from .encoding_guard import encoding_node
from .abuse_detector import abuse_node
from .aggregator import aggregator_node
from .report_builder import report_node

__all__ = [
    "ingest_node", "pii_node", "confidentiality_node",
    "encoding_node", "abuse_node", "aggregator_node", "report_node"
]
```

14. Testing Suite

14.1 tests/conftest.py

```
# tests/conftest.py
import pytest
import json
from unittest.mock import MagicMock, patch

@pytest.fixture
def sample_state():
    return {
        "pdf_path": "/tmp/test.pdf",
        "pdf_name": "test.pdf",
        "upload_id": "test123",
        "total_pages": 2,
        "pages_text": [
            {
                "page_num": 1,
                "text": "Contact John Smith at john.smith@company.com or call +91-9876543210. Aadhaar: 1234 5678 9012",
                "char_count": 100,
                "detected_encoding": "utf-8",
                "encoding_confidence": 0.99,
                "image_count": 0,
            },
            {
                "page_num": 2,
                "text": "API Key: sk-prod-abc123def456xyz789. Password: SuperSecret@123",
                "char_count": 70,
                "detected_encoding": "utf-8",
                "encoding_confidence": 0.99,
                "image_count": 0,
            },
        ],
        "pii_results": [],
        "confidential_results": [],
        "encoding_results": [],
        "abuse_results": [],
        "page_results": [],
        "summary": {},
        "compliance_rules": {
            "pii": {"enabled": True, "min_confidence": 0.75},
            "confidentiality": {"enabled": True, "min_confidence": 0.70, "custom_keywords": []},
            "encoding": {"enabled": True, "allowed_languages": ["en"], "non_ascii_threshold": 5,
                "min_encoding_confidence": 0.85, "flag_non_ascii": True},
            "abuse": {"enabled": True, "zero_tolerance_categories": ["child_safety", "terrorism"]},
        },
    }
```

```

        "report_path": None,
        "processing_complete": False,
        "errors": [],
    }

@pytest.fixture
def mock_ai_pii_response():
    return json.dumps({
        "has_pii": True,
        "findings": [
            {
                "category": "EMAIL",
                "value": "john.smith@company.com",
                "context": "Contact John Smith at john.smith@company.com or call",
                "confidence": 0.99,
                "risk_level": "high"
            },
            {
                "category": "PHONE",
                "value": "+91-9876543210",
                "context": "john.smith@company.com or call +91-9876543210. Aadhaar",
                "confidence": 0.97,
                "risk_level": "high"
            }
        ],
        "page_risk": "high",
        "recommendation": "Redact all PII before sharing"
    })

@pytest.fixture
def mock_ai_clean_response():
    return json.dumps({
        "has_pii": False,
        "findings": [],
        "page_risk": "low",
        "recommendation": "No action required"
    })

```

14.2 tests/test_nodes.py

```

# tests/test_nodes.py
import pytest
import json
from unittest.mock import patch, MagicMock

from pipeline.nodes.encoding_guard import encoding_node
from pipeline.nodes.aggregator import aggregator_node

class TestEncodingNode:
    """Tests for the rule-based encoding guard (no AI calls)."""

    def test_clean_english_text_is_low_risk(self, sample_state):
        result = encoding_node(sample_state)
        assert "encoding_results" in result
        assert len(result["encoding_results"]) == 2

    def test_non_ascii_chars_flagged(self, sample_state):
        sample_state["pages_text"][0]["text"] = "Hello " + "こんにちは" * 10 + " world"
        result = encoding_node(sample_state)
        flags = result["encoding_results"][0]["encoding_flags"]
        flag_types = [f["type"] for f in flags]

```

```

assert "NON_ASCII_CHARS" in flag_types

def test_low_encoding_confidence_flagged(self, sample_state):
    sample_state["pages_text"][0]["encoding_confidence"] = 0.5
    result = encoding_node(sample_state)
    flags = result["encoding_results"][0]["encoding_flags"]
    flag_types = [f["type"] for f in flags]
    assert "LOW_ENCODING_CONFIDENCE" in flag_types

def test_disabled_encoding_returns_empty(self, sample_state):
    sample_state["compliance_rules"]["encoding"]["enabled"] = False
    result = encoding_node(sample_state)
    assert result["encoding_results"] == []

class TestAggregatorNode:
    """Tests for the aggregator node."""

    def test_aggregator_combines_results_correctly(self, sample_state):
        sample_state["pii_results"] = [
            {"page_num": 1, "pii_flags": [{"type": "EMAIL", "confidence": 0.99, "risk_level": "high"}], "pii_risk": "high"},
            {"page_num": 2, "pii_flags": [], "pii_risk": "low"},
        ]
        sample_state["confidential_results"] = [
            {"page_num": 1, "confidential_flags": [], "confidential_risk": "low"},
            {"page_num": 2, "confidential_flags": [{"type": "API_KEY", "confidence": 0.98, "risk_level": "critical"}], "confidential_risk": "critical"}
        ]
        sample_state["encoding_results"] = [
            {"page_num": 1, "encoding_flags": [], "encoding_risk": "low"},
            {"page_num": 2, "encoding_flags": [], "encoding_risk": "low"},
        ]
        sample_state["abuse_results"] = [
            {"page_num": 1, "abuse_flags": [], "abuse_risk": "low"},
            {"page_num": 2, "abuse_flags": [], "abuse_risk": "low"},
        ]

        result = aggregator_node(sample_state)

        assert "page_results" in result
        assert "summary" in result
        assert result["processing_complete"] is True

        # Page 1 should have high risk (PII)
        page1 = next(p for p in result["page_results"] if p["page_num"] == 1)
        assert page1["overall_risk"] == "high"

        # Page 2 should have critical risk (API key)
        page2 = next(p for p in result["page_results"] if p["page_num"] == 2)
        assert page2["overall_risk"] == "critical"

        # Summary
        assert result["summary"]["total_flags"] == 2
        assert result["summary"]["highest_risk"] == "critical"

    def test_empty_results_give_low_risk(self, sample_state):
        sample_state["pii_results"] = [
            {"page_num": 1, "pii_flags": [], "pii_risk": "low"},
            {"page_num": 2, "pii_flags": [], "pii_risk": "low"},
        ]
        sample_state["confidential_results"] = [
            {"page_num": 1, "confidential_flags": [], "confidential_risk": "low"},
            {"page_num": 2, "confidential_flags": [], "confidential_risk": "low"},
        ]
        sample_state["encoding_results"] = [
            {"page_num": 1, "encoding_flags": [], "encoding_risk": "low"},
            {"page_num": 2, "encoding_flags": [], "encoding_risk": "low"}
        ]

```

```

    { "page_num": 2, "abuse_flags": [], "abuse_risk": "low" },
]
sample_state["abuse_results"] = [
    {"page_num": 1, "abuse_flags": [], "abuse_risk": "low"},
    {"page_num": 2, "abuse_flags": [], "abuse_risk": "low"},
]

result = aggregator_node(sample_state)
assert result["summary"]["total_flags"] == 0
assert result["summary"]["highest_risk"] == "low"

```

```
class TestPIINodeWithMockedAI:
```

```
    """Tests for PII node using mocked AI calls."""
```

```

def test_pii_node_detects_email_and_phone(self, sample_state, mock_ai_pii_response):
    with patch("pipeline.nodes.pii_detector.call_ai", return_value=mock_ai_pii_response):
        from pipeline.nodes.pii_detector import pii_node
        result = pii_node(sample_state)

        assert "pii_results" in result
        page1_flags = result["pii_results"][0]["pii_flags"]
        categories = [f["category"] for f in page1_flags]
        assert "EMAIL" in categories
        assert "PHONE" in categories

def test_pii_node_handles_empty_page(self, sample_state, mock_ai_clean_response):
    sample_state["pages_text"][0]["text"] = ""
    with patch("pipeline.nodes.pii_detector.call_ai", return_value=mock_ai_clean_response):
        from pipeline.nodes.pii_detector import pii_node
        result = pii_node(sample_state)

    # Empty page should not call AI
    assert result["pii_results"][0]["pii_flags"] == []

def test_pii_disabled_returns_empty(self, sample_state):
    sample_state["compliance_rules"]["pii"]["enabled"] = False
    from pipeline.nodes.pii_detector import pii_node
    result = pii_node(sample_state)
    assert result["pii_results"] == []

```

14.3 Run Tests

```

# Run all tests
pytest tests/ -v

# Run with coverage report
pip install pytest-cov
pytest tests/ --cov=pipeline --cov=config --cov-report=term-missing

# Run only fast tests (no AI calls)
pytest tests/test_nodes.py -v -k "Encoding or Aggregator"

```

15. Docker Setup

15.1 Dockerfile


```

FROM python:3.11-slim

# Set working directory
WORKDIR /app

# System dependencies for PyMuPDF and ReportLab
RUN apt-get update && apt-get install -y \
    gcc \
    libmupdf-dev \
    libfreetype6-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first (cache layer)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create necessary directories
RUN mkdir -p reports storage

# Expose Streamlit port
EXPOSE 8501

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8501/_stcore/health || exit 1

# Run Streamlit
CMD ["streamlit", "run", "app/main.py", \
    "--server.port=8501", \
    "--server.address=0.0.0.0", \
    "--server.headless=true", \
    "--browser.gatherUsageStats=false"]

```

15.2 docker-compose.yml

```

version: "3.9"

services:
  compliance-scanner:
    build: .
    container_name: pdf-compliance-scanner
    ports:
      - "8501:8501"
    volumes:
      - ./reports:/app/reports          # Persist generated reports
      - ./storage:/app/storage          # Persist SQLite database
      - ./config/rules.json:/app/config/rules.json # Persist rules
    environment:
      - GROQ_API_KEY=${GROQ_API_KEY}
      - AI_PROVIDER=${AI_PROVIDER:-groq}
      - GROQ_MODEL=${GROQ_MODEL:-llama3-70b-8192}
    env_file:
      - .env
    mem_limit: 2g
    restart: unless-stopped

```

15.3 Run with Docker

```
# Build and run
docker-compose up --build

# Run in background
docker-compose up -d

# View logs
docker-compose logs -f

# Stop
docker-compose down
```

16. GitHub Actions CI/CD

16.1 .github/workflows/ci.yml

```
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest-cov

      - name: Run tests
        run: |
          pytest tests/ -v --cov=pipeline --cov=config \
            --cov-report=xml --cov-report=term-missing
        env:
          GROQ_API_KEY: ${ secrets.GROQ_API_KEY }
          AI_PROVIDER: groq

      - name: Upload coverage report
        uses: codecov/codecov-action@v4
        with:
          file: ./coverage.xml
          fail_ci_if_error: false

  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```

- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: "3.11"
- name: Install bandit
  run: pip install bandit safety
- name: Run security scan
  run: |
    bandit -r pipeline/ config/ app/ -ll
    safety check -r requirements.txt

docker-build:
  runs-on: ubuntu-latest
  needs: test
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - name: Build Docker image
      run: docker build -t pdf-compliance-scanner .

```

16.2 Add Secrets to GitHub

1. Go to your GitHub repo
2. **Settings** → **Secrets and variables** → **Actions**
3. Click **New repository secret**
4. Add:
 - Name: `GROQ_API_KEY` | Value: your Groq API key

17. Deployment to Streamlit Cloud (Free)

17.1 Push Everything to GitHub

```

# From your project root
git add .
git commit -m "feat: complete PDF compliance scanner implementation"
git push origin main

```

17.2 Create secrets.toml for Cloud (DO NOT commit this)

Create `.streamlit/secrets.toml` locally (add to `.gitignore`):

```

GROQ_API_KEY = "gsk_your_actual_key_here"
AI_PROVIDER = "groq"
GROQ_MODEL = "llama3-70b-8192"

```

Add to `.gitignore`:

```

.env
.streamlit/secrets.toml
*.db
reports/
__pycache__/
.venv/

```

17.3 Deploy on Streamlit Community Cloud

1. Go to <https://share.streamlit.io>
2. Sign in with your GitHub account
3. Click **New app**
4. Select your repository: `pdf-compliance-scanner`
5. Branch: `main`
6. Main file path: `app/main.py`
7. Click **Advanced settings**

8. In **Secrets** paste:

```
GROQ_API_KEY = "gsk_your_key_here"
AI_PROVIDER = "groq"
GROQ_MODEL = "llama3-70b-8192"
```

9. Click **Deploy!**

Your app will be live at: `https://your-app-name.streamlit.app`

17.4 Update the App

```
git add .
git commit -m "update: ..."
git push origin main
# Streamlit Cloud auto-redeploys in ~1 minute
```

18. Alternative: Deploy to Hugging Face Spaces (Free)

18.1 Create a Space

1. Go to <https://huggingface.co/new-space>
2. Space name: pdf-compliance-scanner
3. Select **Streamlit** as the SDK
4. Select **Free** tier
5. Create space

18.2 Push Code to HF Space

```
# Add HF remote (replace YOUR_USERNAME)
git remote add hf https://huggingface.co/spaces/YOUR_USERNAME/pdf-compliance-scanner

# Push (you'll need HF token with write access)
git push hf main
```

18.3 Add Secrets in HF Spaces

1. Go to your Space → **Settings**
2. Under **Repository secrets** click **New secret**
3. Add `GROQ_API_KEY` with your key

19. Troubleshooting & Common Errors

Error: `ModuleNotFoundError: No module named 'fitz'`

```
pip install PyMuPDF==1.24.5
```

Error: `Groq rate limit (429)`

The free tier allows 30 requests/minute. Add `time.sleep(2)` between page calls, or switch to `llama3-8b-8192` (faster, higher limits).

Error: `langdetect: No features found`

Occurs on very short text. The encoding node handles this — it only runs language detection on text > 100 chars.

Error: `JSON decode error from AI response`

The `parse_json_response()` function in `config/ai_provider.py` handles this by stripping markdown fences. If it still fails, add a try/except fallback in the node.

Error: `LangGraph state key mismatch`

Every node must return a **partial dict** with keys that exist in `PipelineState` . Check for typos in return keys.

Streamlit Cloud error: `ModuleNotFoundError`

Ensure `requirements.txt` is in the **root** of your repo (not inside a subfolder).

ReportLab font warnings

```
pip install reportlab[rl_package]
```

Ollama local setup (alternative to Groq):

```
# Install Ollama
curl https://ollama.ai/install.sh | sh

# Pull a model
ollama pull llama3

# Start Ollama server
ollama serve

# In .env set:
AI_PROVIDER=ollama
OLLAMA_MODEL=llama3
```

📋 Final Checklist Before Submission

- ☐ GitHub repo is public with complete README
- ☐ All 4 compliance checks implemented and working
- ☐ Streamlit app deployed and accessible via URL
- ☐ Rules editor updates rules in real-time
- ☐ PDF compliance report downloads correctly
- ☐ `.env` not committed (secrets only in `.env.example`)
- ☐ Docker container builds and runs with `docker-compose up`
- ☐ All tests pass with `pytest tests/ -v`
- ☐ GitHub Actions CI passes (green checkmark on repo)
- ☐ Demo PDF ready with all 4 violation types

📄 Creating a Demo PDF with Violations

Use this Python script to create a test PDF with intentional violations:

```
# create_test_pdf.py – run this to generate a demo PDF for testing
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4

def create_test_pdf():
    c = canvas.Canvas("tests/fixtures/demo_violations.pdf", pagesize=A4)
    width, height = A4

    # Page 1: PII violations
    c.setFont("Helvetica-Bold", 14)
    c.drawString(50, height - 50, "Employee Record – Confidential")
    c.setFont("Helvetica", 11)
    c.drawString(50, height - 90, "Name: John Smith")
    c.drawString(50, height - 110, "Email: john.smith@company.com")
    c.drawString(50, height - 130, "Phone: +91-9876543210")
    c.drawString(50, height - 150, "Aadhaar: 1234 5678 9012")
    c.drawString(50, height - 170, "Address: 42 MG Road, Bangalore 560001")
    c.showPage()

    # Page 2: Confidential data
    c.setFont("Helvetica-Bold", 14)
    c.drawString(50, height - 50, "System Configuration – DO NOT SHARE")
    c.setFont("Helvetica", 11)
    c.drawString(50, height - 90, "API Key: sk-prod-abc123def456ghi789jkl012mno345pqr")
    c.drawString(50, height - 110, "Database password: SuperSecret@DB123")
    c.drawString(50, height - 130, "Project: OPERATION PHOENIX – Phase 2 Launch")
    c.drawString(50, height - 150, "Q4 Revenue Target: $42M (confidential)")
    c.showPage()

    # Page 3: Clean page
    c.setFont("Helvetica-Bold", 14)
    c.drawString(50, height - 50, "Public Information")
    c.setFont("Helvetica", 11)
    c.drawString(50, height - 90, "Our company was founded in 2020.")
    c.drawString(50, height - 110, "We operate in the technology sector.")
    c.drawString(50, height - 130, "Contact our PR team for media inquiries.")
    c.showPage()

    c.save()
    print("Demo PDF created: tests/fixtures/demo_violations.pdf")

create_test_pdf()
```

```
python create_test_pdf.py
```